

[LAB-06] 데이터 시각화 통합 정리 (matplotlib / seaborn / my_plot 모듈)

이 문서는 LAB-06 동안 정리한 7개 문서(그래프 기본 구조, Line/KDE/Hist/Box, 레퍼런스 가이드, Scatter/ConvexHull, Lm/Pair Plot, SubPlot/Twin Axis, Violin/Heatmap/Bar/Pie)를 **하나로 합치고 중복을 제거한 최종본**이다. 목적: 새 데이터셋으로 시각화 작업할 때 이 문서 하나만 펴놓고 개념 확인 + 코드 복붙이 다 되게 하는 것. 그래서 개념 설명과 실전 코드를 최대한 다 남겼다.

목차

1. 환경 세팅 (폰트, rcParams)
2. 진짜 기초 개념 — `plt`, `fig`, `ax`란 무엇인가
3. 그래프 기본 4단계 템플릿
4. `my_plot` 모듈 설계 철학 (핵심)
5. seaborn 공통 호출 패턴 & hue/palette/color 상호작용 규칙
6. Long ↔ Wide 포맷 변환 (melt)
7. 차트별 정리 (LinePlot / KDE / Histogram / BoxPlot:ViolinPlot / Heatmap / Bar-Count / Pie-Donut / Stackplot)
8. Scatter Plot & Convex Hull
9. LmPlot & PairPlot (figure-level 함수)
10. SubPlot (서브플롯)
11. Twin Axis (이중 축)
12. `my_plot` 모듈 함정 노트 (버그성 주의사항)
13. EDA 파이프라인 — `my_qtcheck` 모듈
14. 새 데이터셋 적용 시 종합 체크리스트
15. 자주 하는 실수 모음 (통합 오답노트)
16. 실전 템플릿 모음 (바로 복붙용)
17. 색상 팔레트 참고

1. 환경 세팅 (폰트, rcParams)

노트북 맨 위에서 **세션당 한 번만** 실행하면 되는 부분. 그래프 하나 그릴 때마다 반복할 필요 없음.

1-1. 한글 폰트 설정

matplotlib 기본 폰트는 한글 미지원 → 한글 쓰면 네모(□)로 깨짐. 아래 순서로 등록:

```
from matplotlib import font_manager as fm

fpath = "../helpers/fonts" # 상대경로 주의! (아래 체크리스트 참고)
font_files = gl.glob(os.path.join(fpath, "*.ttf"))

for f in font_files:
```

```

fm.fontManager.addfont(f)          # matplotlib에 폰트 등록
fprop = fm.FontProperties(fname=f)  # 폰트 속성 읽기
fname = fprop.get_name()           # 폰트 "이름" 추출
plt.rcParams['font.family'] = fname # 전역 기본 폰트로 지정

```

`my_plot.py` 모듈 안에서는 이 과정이 OS별로 자동 분기되어 있어 신경 안 써도 됨:

Windows → 'Malgun Gothic', Mac → 'AppleGothic', Linux → 'NanumGothic'

재사용 시 체크리스트

- `fpath`는 **상대경로**라서 노트북 파일 위치가 바뀌거나 다른 프로젝트에서 복붙하면 `font_files`가 빈 리스트가 되고 폰트 등록이 **조용히 실패함** (에러 안 남!). → 새 프로젝트에서 쓸 땐 `os.getcwd()`로 현재 작업 폴더 확인 후 `fpath`를 절대경로 또는 올바른 상대경로로 고칠 것.
- `rcParams['font.family']`는 **전역 설정**이라 같은 커널 세션 내내 유지됨. 커널을 껐다 켜면 초기화되므로 새 세션에서는 다시 실행해야 함.

1-2. 그래프 기본 옵션 (`rcParams`)

```

my_dpi = 200
plt.rcParams['font.size'] = 12          # 기본 글자 크기
plt.rcParams['axes.unicode_minus'] = False # 마이너스(-) 부호 깨짐 방지
plt.rcParams['figure.dpi'] = my_dpi     # 화면 표시 해상도
plt.rcParams['savefig.dpi'] = my_dpi    # 파일 저장 해상도
plt.rcParams['lines.linewidth'] = 2     # 선 굵기 기본값
plt.rcParams['axes.axisbelow'] = True   # 격자선을 그래프 뒤로

```

- `axes.unicode_minus = False` 안 하면 음수 축 눈금이 네모로 깨질 수 있음 (한글 폰트 특성).
- `axes.axisbelow = True`는 잘 빼먹는데, 없으면 grid 선이 막대/점 위에 그려져 지저분해짐.
- `figure.dpi`와 `savefig.dpi`를 같은 변수(`my_dpi`)로 맞춰서 **화면 해상도 = 저장 파일 해상도** 습관을 들일 것.

2. 진짜 기초 개념 — `plt`, `fig`, `ax`란 무엇인가

여기부터 확실히 잡으면 뒤 내용이 다 쉬워진다.

(1) `plt`

```
from matplotlib import pyplot as plt
```

`matplotlib`이라는 그림 라이브러리의 "그래프 그리는 도구 모음". 비유: **`plt` = 미술 도구함 전체** (캔버스, 붓, 액자 다 들어있음).

(2) `fig`

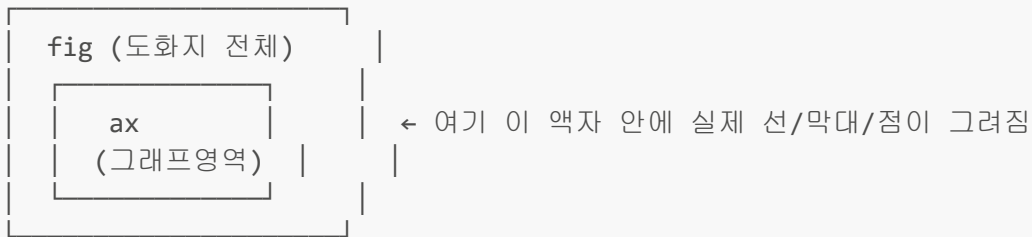
fig는 도화지 그 자체. 그림이 그려질 종이 한 장.

```
fig, ax = plt.subplots()
```

이 한 줄: "도화지 한 장(**fig**) 줘. 그 위에 그림 그릴 영역(**ax**)도 하나 줘."

(3) **ax** (제일 핫갈리는 부분)

ax는 **axes**의 줄임말. 도화지 안에서 실제로 그래프가 그려지는 네모난 영역 하나.



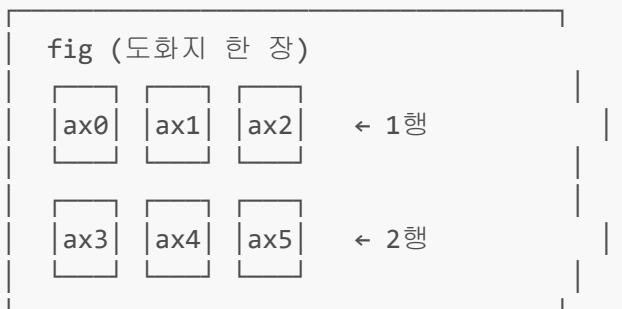
- **fig** = 벽에 걸 액자 틀 전체
- **ax** = 그 액자 틀 안의 "그림이 실제로 그려지는 캔버스"

축, 눈금, 제목, 그리드, 실제 선/막대 — 이 모든 게 **ax** 위에 그려진다. **fig** 자체에는 그림이 직접 그려지지 않는다. 그래서 **sb.boxplot(..., ax=???)**처럼 항상 "어느 **ax**에 그릴지"를 지정해주는 것.

(4) 왜 도화지(**fig**)와 그림영역(**ax**)을 따로 나눴나

한 도화지 안에 그림영역을 여러 개 넣을 수 있게 하려고.

```
fig, ax = plt.subplots(2, 3) # 도화지 한 장 + 2행 3열 그림영역 6개
```



이게 바로 서브플롯의 정체.

(5) **ax.flatten()**은 왜 하는가

`rows>1` 또는 `cols>1`이면 `ax`는 2차원 배열로 저장됨 (`ax[0][0]`, `ax[0][1]`, `ax[1][0]`... 식으로 접근해야 함 → 번거로움).

`flatten()`은 이 2차원 표를 한 줄로 펴줌:

```

퍼기 전:      퍼고 난 후:
[ax0 ax1 ax2]
[ax3 ax4 ax5] → [ax0, ax1, ax2, ax3, ax4, ax5]

```

그러면 `ax[0]`, `ax[1]`...로 순서대로(왼쪽→오른쪽, 위→아래, row-major) 꺼내 쓸 수 있어서 반복문 돌리기 편해짐.

(6) 전체 그리기 흐름 연결

```

fig, ax = plt.subplots(2, 3) # ① 도화지 + 액자 6개 준비
ax = ax.flatten()           # ② 순서대로 꺼내 쓸 수 있게 정리

sb.boxplot(data=df, ax=ax[0]) # ③ 1번째 액자
sb.histplot(data=df, ax=ax[1]) # ④ 2번째 액자
...

plt.show()                  # ⑤ 도화지 전체를 화면에 표시

```

- `fig`는 딱 한 번, `show()`도 딱 한 번만 호출.
- `ax`는 여러 개, 그리기 함수 호출할 때마다 "몇 번째 `ax`에 그릴지" 지정.

(7) axes-level 함수 vs figure-level 함수 (매우 중요)

- **axes-level:** `sb.lineplot`, `sb.barplot`, `sb.scatterplot`, `sb.histplot`, `sb.boxplot`, `sb.violinplot`, `sb.kdeplot`, `sb.heatmap` 등 → `ax=` 파라미터를 받음 → 내가 만든 `ax` 위에 그릴 수 있음 → **모듈화하기 쉬움**, 서브플롯/이중축 둘 다 가능.
- **figure-level:** `sb.lmplot`, `sb.pairplot`, `sb.catplot`, `sb.relplot`(FacetGrid 기반) → `ax=`를 안 받고 **자기**가 알아서 새 **Figure**를 만들어버림 → 내가 만든 `ax`에 못 끼워 넣음. 서브플롯의 한 칸에도, 이중축에도 못 들어감. 결과로 `g`라는 **FacetGrid** 객체가 반환되며, `g.fig`(figure 객체), `g.axes.flatten()`(내부 `ax`들)으로 조작.

⚠ 서브플롯 계획 짤 때 `lmplot/pairplot`은 처음부터 "따로 그릴 것"으로 빼고 칸 수를 계산해야 함. 나중에 서브플롯에 넣고 싶으면 이미지로 저장 후 삽입하는 등 별도 처리가 필요함 (현재 구조에서는 지원 안 됨).

3. 그래프 기본 4단계 템플릿 (제일 중요한 개념)

```

# 1) 그래프 초기화
width, height = 1280, 640
rows, cols = 1, 1
my_figsize = (width/100, height/100) # px → inch 변환 (100은 관례적 기준)
fig, ax = plt.subplots(rows, cols, figsize=my_figsize, dpi=my_dpi)
ax.grid(True, alpha=0.5)

```

```
# 2) 옵션 설정 (제목/축 라벨)
ax.set_title("제목", color="#6a91ca", fontsize=24, fontweight=700, pad=15)
ax.set_xlabel("X축", color="#ff6600", fontsize=16, fontweight=500, labelpad=5)
ax.set_ylabel("Y축", color="#ff00ff", fontsize=16, fontweight=500, labelpad=5)

# 3) 그래프 그리기 ← 여기에 seaborn 함수 (lineplot, barplot 등)
# 예: sb.lineplot(data=df, x=..., y=..., ax=ax)

# 4) 출력
plt.tight_layout()
plt.savefig("myplot.png", dpi=my_dpi)
plt.show()
plt.close()
```

단계	하는 일	빠지면 생기는 문제
1) 초기화	<code>fig</code> , <code>ax</code> 생성	<code>ax</code> 없으면 뒤 코드 실행 자체가 안 됨
2) 옵션 설정	제목/축 이름 등 꾸미기	없어도 그려지지만 뭘 보여주는지 알 수 없음
3) 그리기	실제 데이터를 <code>ax</code> 위에 시각화	빠지면 빈 액자만 남음
4) 출력	저장 + 화면 표시 + 메모리 정리	<code>plt.close()</code> 안 하면 그래프 객체가 메모리에 계속 쌓임 (여러 개 그릴 때 느려지는 원인)

→ 이 4단계는 그래프 종류와 무관하게 항상 같은 뼈대다. 3단계(그리기)에 들어가는 함수만 `lineplot`/`barplot`/`scatterplot` 등으로 바뀌고 나머지는 고정 → 그래서 이것 함수로 뽑아낸 게 `my_plot` 모듈.

4. `my_plot` 모듈 설계 철학 (핵심)

4-1. 모듈화의 목적

- **반복 제거:** `init()` → 그리기 → `show()` 골격은 그래프 종류가 달라도 항상 똑같이 반복됨. 손으로 매번 치면 오타/누락 실수가 생김.
- **일관성:** 프로젝트 전체에서 폰트 크기, 그리드 스타일, 제목 위치가 항상 동일하게 나옴 (한 군데서 관리).
- **재사용:** 새 데이터셋에도 `my_plot.lineplot(data=새데이터, x=..., y=...)` 한 줄로 끝남.

4-2. 뼈대: `init()` + `show()`

```
def init(width=1280, height=640, rows=1, cols=1, title=None, xlabel=None,
ylabel=None, grid=True, twinx=False):
    # figsize, title/label, grid까지 세팅한 fig, ax 반환
    ...
def show(save_path=None):
    # tight_layout → show → close (save_path 있으면 저장까지)
    ...
```

lineplot, kdeplot, histplot, boxplot, violinPlot, heatmap, barplot, countplot, pieplot, stackplot, scatterplot 전부 이 두 함수를 공통으로 사용.

4-3. ax=None 패턴 — 모듈 설계의 핵심 아이디어

```
def 어떤plot(data=None, x=None, y=None, ..., title=None, xlabel=None, ylabel=None,
             width=1280, height=640, save_path=None, ax=None):
    fig = None
    if ax is None:
        fig, ax = init(width=width, height=height, title=title, xlabel=xlabel,
                      ylabel=ylabel)

    sb.어떤plot(data=data, x=x, y=y, ..., ax=ax)  # 실제 그리기는 seaborn에 위임

    if fig is not None:
        show(save_path=save_path)
```

상황	ax=None (직접 새로 만들)	ax=넘겨받음 (서브플롯/이중축의 한 칸)
동작	함수가 직접 fig/ax 생성	넘겨받은 ax에 그냥 그림
show()	호출됨 (그래프 하나만 단독 출력)	호출 안 됨 (바깥에서 전체 다 그린 후 한 번만)
예시	my_plot.lineplot(data=df, x='년도', y='건수')	my_plot.lineplot(data=df, x='년도', y='건수', ax=ax[0])

왜 이렇게 짜야 하는가 — 새 그래프 함수 만들 때마다 스스로 물어볼 질문:

1. 이 함수, 서브플롯 안에서도 쓰일 수 있어야 하나? → ax=None을 마지막 파라미터로 추가하고 이 패턴 적용.
2. ax가 넘어왔을 때 내부에서 새 fig를 만들면 안 됨 → init() 호출 대신 넘겨받은 ax에 그리기만.
3. show() 호출은 "내가 fig를 직접 만들었을 때만" → 안 그러면 서브플롯 중간에 화면이 조각조각 튀어나옴.

- seaborn 호출부에 ax=ax를 빠뜨리면 새 figure가 생기거나(figure-level) 엉뚱한 축에 그려짐 → 함수 만들 때마다 체크.

→ 하나의 함수가 "단독 그래프"와 "서브플롯/이중축의 한 조각" 역할을 다 할 수 있게 설계된 것. 이게 재사용성 확장의 핵심.

4-4. axes-level만 이 패턴을 따를 수 있음 (예외: Implot/pairplot)

- init()은 plt.subplots() 기반(axes-level)인데, lmpot/pairplot은 자기가 알아서 새 Figure를 만드는 figure-level 함수라서 ax를 미리 받아 꽃을 수 없음.
- 그래서 my_plot.lmpot(), my_plot.pairplot()은 ax=None 분기 없이 항상 자체적으로 그리고 항상 show()까지 호출하는 별도 구조.
- 새 함수를 모듈에 추가할 때, axes-level인지 figure-level인지부터 먼저 확인해야 설계 패턴이 정해짐.

4-5. 파라미터 설계 원칙

- seaborn 원래 파라미터명을 최대한 그대로 사용 (hue, palette, markers 등) → seaborn 문서 그대로 재사용 가능.

- 중첩된 `scatter_kws={"edgecolor":..., "s":...}` 같은 건 `scatter_edgecolor`, `scatter_size`처럼 **평평하게(flatten)** 풀어서 호출 → 딕셔너리 문법 없이 호출 가능.
- 조건부 무효화 로직: `if not hue and palette: palette=None` → hue 없이 palette만 준 무의미한 조합을 자동 정리 (사용자 실수 방지).

4-6. `init()/show()`가 대신해주는 것 vs 함수 안에서 다시 해줘야 하는 것

`title`, `xlabel`, `ylabel`, `width`, `height`는 `init()` 호출 시점에만 적용됨. 나중에 `ax=ax`를 다른 그리기 함수에 넘기면서 `title` 등을 바꾸고 싶으면 **그 함수 안에서 다시 `ax.set_title(...)`을 호출해야 함** — `init()`을 또 부른다고 적용되는 게 아님. (자세한 함정은 12장 참고.)

4-7. 함수 이름 겹치지 않게 관리

같은 이름의 함수를 실수로 두 번 정의하면 나중에 정의한 게 앞의 걸 조용히 덮어씀 (에러 없음) → 함수 늘어날수록 이름 규칙(`plot_line`, `plot_bar` 등 접두어 통일)을 정해두는 게 좋음.

4-8. 커널 캐시 문제 — 모듈 수정할 때 자주 걸리는 함정

`helpers/my_plot.py`를 수정하고 저장해도, 이미 `import`된 주피터 커널은 **예전 버전을 메모리에 들고 있음**. → 반영이 안 될 때:

- `importlib.reload(my_plot)` 사용, 또는
- 커널 재시작(Restart) 후 처음부터 재실행. → "코드는 고쳤는데 왜 안 바뀌지?" 싶을 때 제일 먼저 의심할 것.

4-9. "직접 코딩" vs "모듈 사용" — 언제나 반복되는 두 버전 비교

	직접 코딩 버전 (raw)	모듈 사용 버전 (my_plot)
코드량	길다	짧다 (1~2줄)
내부 동작 확인	눈에 다 보임 → 원리 이해용	안 보임 → 모듈 안에 숨어있음
커스터마이징	그 자리에서 자유롭게 수정	모듈 파일을 고쳐야 함
용도	처음 배울 때 / 원리 이해할 때	원리 이해 끝난 후 / 실전에서 빠르게 그릴 때

사고 순서 (중요): 새 옵션-그래프 종류를 배우면 → ① **직접 코딩**으로 셀 하나하나 실행하며 원리 확인 → ② 검증되면 **모듈에 함수로 캡슐화** → ③ 이후엔 모듈 버전으로 빠르게 그림. **모듈화의 이득은 "코드가 짧아진다"가 아니라 "매번 같은 실수를 할 여지가 없어진다"는 것.** (`fontsize=32`를 매번 손으로 치면 그래프마다 미세하게 어긋날 수 있는데, 함수로 고정하면 그럴 일이 없음.) → 순서가 뒤집히면 안 됨. 원리 모른 채 모듈부터 만들면 나중에 모듈이 안 될 때 왜 안 되는지 못 찾음.

5. seaborn 공통 호출 패턴 & hue/palette/color 상호작용 규칙

5-1. 모든 seaborn plot 함수의 3가지 호출 패턴 (공통)

패턴	코드 예시	설명
자동인식	<code>sb.lineplot([10,20,30,40,50])</code>	리스트 하나만 넘기면 y값으로 인식, x는 인덱스

패턴	코드 예시	설명
개별 데이터 지정	<code>sb.lineplot(x=[...], y=[...])</code>	x, y를 Series/리스트로 따로 지정
DataFrame + 컬럼명 ✅	<code>sb.lineplot(data=df, x='col1', y='col2')</code>	실무 권장 — 모든 plot 함수와 인터페이스 통일됨

`lineplot`, `barplot`, `boxplot`, `scatterplot`, `violinplot` 등 전부 `data=df`, `x='컬럼명'`, `y='컬럼명'` 패턴으로 통일 → 나중에 함수 갈아끼우기 쉬움 (일반화의 핵심).

5-2. hue / palette / color 상호작용 규칙 (여러 함수에서 반복되는 공식)

- **hue가 있으면 color는 무의미해짐** — hue가 색상을 결정하는 주체가 되기 때문. (`scatterplot`, `kdeplot`의 `meanline` 등 여러 함수에서 반복되는 패턴.)
- **palette는 반드시 hue와 함께 써야 적용됨.** hue 없이 palette만 쓰면 무시되거나 경고가 남 (`barplot` 등).
- `my_plot` 모듈 내부 방어 로직 예시:

```
if hue == None:
    if color is None and palette is not None:
        color = sb.color_palette(palette)[0]
        palette = None
else:
    color = None
```

→ "군집이 없는데 색을 나눠 그릴 수 없다"는 당연한 전제를 조건문으로 명시한 것. 옵션 하나가 다른 옵션의 존재를 전제로 할 때 방어 조건을 걸어두는 게 좋은 설계.

6. Long ↔ Wide 포맷 변환 (melt) — 매우 중요

6-1. 왜 필요한가

seaborn은 **hue로 범주를 나누려면 "긴 형식(long format)" 데이터가 필요하다.**

- Wide format: 서울시 일일 확진, 전국 일일 확진처럼 변수가 **컬럼**으로 따로 존재.
- Long format: 구분(변수명), 확진자 수(값) 컬럼 2개로 합쳐진 형태.

이렇게 바뀌야 `hue='구분'` 하나로 여러 선/막대를 자동으로 나눠 그릴 수 있다.

6-2. melt 사용법

```
df2 = melt(df, id_vars='기준일',
            value_vars=['서울시 일일 확진', '전국 일일 확진'],
            var_name='구분', value_name='확진자 수')
```

- `id_vars`: 고정시킬 기준 컬럼 (날짜처럼 펼쳐지 않을 컬럼)
- `value_vars`: 세로로 풀어낼(녹일) 컬럼들
- `var_name`: 원래 컬럼명이 들어갈 새 컬럼 이름

- `value_name`: 원래 값이 들어갈 새 컬럼 이름

`melt` 전후 shape 변화: (N 행, 3열) $\rightarrow$ ($N \times 2$ 행, 3열) (컬럼 2개가 행으로 풀리면서 행 수가 2배).

주의: `id_vars`, `value_vars`를 지정하지 않고 `origin.melt()`처럼 인자 없이 호출하면 모든 컬럼이 의도와 다르게 `variable/value`로 풀려버림 $\rightarrow$ 항상 명시할 것. 또한 **`melt` 결과는 반드시 새 변수에 저장한 뒤** 그 변수로 후속 작업을 해야 함 (원본 `origin`에 대고 `groupby` 등을 시도하면 에러).

6-3. 실전 활용 — bar/count plot에 melt 적용

```
df = melt(origin, id_vars='종사자규모별', var_name='년도', value_name='종사자 수')
my_plot.barplot(data=df, x="종사자규모별", y="종사자 수", hue="년도", ...)
```

$\rightarrow$ "연도별 컬럼이 옆으로 넓게 퍼진(wide) 원본"을 `barplot`에 바로 못 넣을 때, `melt()`로 세로 형태(long)로 바뀌어서 `hue`에 태우는 실무 필수 패턴.

7. 차트별 정리

7-1. LinePlot — 추세/시계열

```
my_plot.lineplot(data=df, x="날짜", y="매출", hue="지역",
                 linewidth=2.0, linestyle="-", marker="o", markersize=6)
```

raw seaborn 스타일 옵션:

```
sb.lineplot(data=origin, x='Date', y='Quantity',
            c="#1D84C3",           # 선 색상
            linestyle=':',         # 점선 (':' 점점선, '--' 점선, '-' 실선)
            linewidth=5,          # 선 굵기
            marker="o",           # 마커 모양
            markersize=20,        # 마커 크기
            markeredgewidth=3,    # 마커 테두리 굵기
            markerfacecolor="#00ff00", # 마커 안쪽 색
            markeredgewidth=3,    # 마커 테두리 색)
```

축 범위/눈금 설정

- `ax.set_ylim(18, 28)`: y축 범위 제한 (`set_xlim`도 동일)
- `ax.set_yticks(눈금위치리스트, 눈금에표시할값리스트, fontsize=6, rotation=-45)` — "실제 위치"와 "표시할 텍스트"를 분리 가능

다중 선 그래프 (같은 좌표축에 여러 선)

```
sb.lineplot(x=[1,2,3,4], y=[10,20,35,40], label="A변화량")
sb.lineplot(x=[1,2,3,4], y=[10,15,30,35], label="B변화량")
ax.legend(loc='upper left')
```

→ `lineplot`을 두 번 호출하면 같은 ax 위에 겹쳐 그려짐. `label` 지정해야 범례가 의미 있게 표시됨.

7-2. KDE Plot (커널밀도추정)

히스토그램을 부드러운 곡선으로 추정한 것. 막대 대신 연속적인 확률밀도 곡선으로 분포를 표현.

- `sb.kdeplot(data=df, x='컬럼')` : 변수 하나의 분포
- `sb.kdeplot(data=df)` (x, y 미지정) : 데이터프레임의 모든 연속형 변수의 밀도 곡선을 한 번에 그림
- x 대신 y 지정 시 세로 방향
- **이변량 KDE**: `sb.kdeplot(data=origin, x="Petal.Width", y="Sepal.Width", hue="Species")`
→ 두 변수의 결합분포(등고선)

평균선 추가

```
mean_val = origin["Sepal.Width"].mean()
ax.axvline(x=mean_val, linestyle='-', color='#ff2200') # 세로선
ax.text(x=mean_val+0.05, y=ax.get_ylim()[1]*0.95,
        s=f'Mean: {mean_val:.2f}', color='#ff2200', fontsize=14)
```

- `axvline`: x좌표에 수직선
- `text(x, y, s=...)`: 임의 좌표에 텍스트. `ax.get_ylim()[1]`은 현재 y축 최댓값 → 그래프 높이에 비례한 위치에 자동 배치하는 트릭

모듈화 버전

```
my_plot.kdeplot(data=df, x="가격", hue="등급", fill=True, meanline=True)
```

- `fill=True`: 곡선 아래 면적 채움
- `meanline=True`: 평균선 자동 표시 (단, `hue` 있으면 범주별 평균선 — 조합 제약 있음, hue/color 상호배타 원칙 동일 적용)

7-3. Histogram

구간(bins) 직접 계산

```
hist, bins = np.histogram(origin['월수입'], bins=5)
bins = np.round(bins, 1)
```

- `np.histogram`은 반드시 숫자형(연속형) 데이터에만 사용 가능. ❌ 범주형(문자열) 컬럼에 쓰면 안 됨 (범주형 빈도는 `value_counts()` + `sb.barplot()`/`sb.countplot()` 사용).

- 반환값: `hist`(각 구간 빈도수), `bins`(구간 경계값, 구간개수+1개)
- 이 `bins`를 `sb.histplot(..., bins=bins)`에 그대로 넣으면 x축 구간이 정확히 일치함

kde 겹치기: `sb.histplot(data=df, x='col', bins=bins, kde=True)` → 히스토그램 위에 KDE 곡선 겹쳐 표시

모듈화 버전

```
my_plot.histplot(data=df, x="가격", bins=20, hue="등급", kde=True)
```

- `bins`: 정수(구간 개수) 또는 리스트(직접 경계 지정)

7-4. BoxPlot / ViolinPlot — 그룹별 분포 비교

개념 차이: Violin Plot = Box Plot(사분위수) + KDE(밀도 분포)를 합친 것. Box Plot은 ==중앙값==·사분위·이상치는 보여주지만 "그 안에서 데이터가 어떻게 몰려있는지(단봉/쌍봉 등)"는 못 보여줌 → Violin이 보완.

boxplot 호출 패턴

- `sb.boxplot(data=series, orient="h")`: 가로 방향
- `sb.boxplot(data=df, orient="v")`: DataFrame 전체 컬럼을 세로로 비교
- `sb.boxplot(data=df, y='컬럼') / x='컬럼'`: 단일 컬럼 (y면 세로, x면 가로)
- `sb.boxplot(data=df, y='컬럼', hue='범주')`: 범주별 비교

orient 규칙: 데이터를 **x**에 지정하면 자동 **가로(h)**, **y**에 지정하면 자동 **세로(v)**. 명시적으로 `orient="h"/"v"` 강제 지정도 가능.

violinplot 데이터 지정 5가지 패턴 (전부 같은 그래프의 다른 방법)

패턴	코드	특징
1. 자동 인식 (Series)	<code>sb.violinplot(data=origin['중간고사'], orient="h")</code>	컬럼 하나를 Series로 직접 전달
2. 자동 인식 (DataFrame)	<code>sb.violinplot(data=origin, orient="v")</code>	DataFrame 전체 → 숫자형 컬럼들이 나란히
3. 개별 데이터 직접 지정	<code>sb.violinplot(x=df['기말고사'])</code>	x에 넣으면 가로 방향
4. DataFrame + 컬럼명	<code>sb.violinplot(data=origin, y="중간고사")</code>	컬럼명 문자열 지정 (가독성 good)
5. 범주(hue) 지정	<code>sb.violinplot(data=origin, y='중간고사', hue='전공', palette='Set2')</code>	그룹별 분포 비교

분석 순서: 처음엔 컬럼 하나(패턴1~4)로 전체 분포 확인 → 이후 집단 간 비교 필요할 때 **hue** 추가(패턴5)로 확장하는 게 자연스러움.

모듈화 버전

```
my_plot.boxplot(data=df, x="등급", y="가격", hue="색상", orient='v')
my_plot.violinPlot(data=df, x="등급", y="가격", orient='v')
my_plot.violinPlot(data=origin, y='중간고사', hue='전공', ylabel="중간고사",
                    width=800, height=480, palette='pastel')
```

- **orient**: 'v'(세로) / 'h'(가로)
- **boxplot**: ==중앙값==사분위-이상치를 정확한 값으로 보고 싶을 때. **violin**: 분포의 전체 모양(다봉/단봉 등)까지 보고 싶을 때.

실전 패턴 — 그룹별 이상치 제거 후 비교 (cookie_cats 예제)

```
# 1) 그룹별로 나눠서
df_a = origin[origin['version'] == 'A']
df_b = origin[origin['version'] == 'B']

# 2) 그룹별 numerical_summary()로 IQR 경계를 각각 구하고
desc_df = my_qtcheck.numerical_summary(df_a)

# 3) 그 경계로 각 그룹의 이상치를 제거한 뒤
df_a_qt = df_a[(df_a['col'] >= desc_df['lower_bound']['col']) &
                (df_a['col'] <= desc_df['upper_bound']['col'])]

# 4) 다시 합쳐서 (index는 원본 유지하려면 ignore_index=False)
df_qtcheck = concat([df_a_qt, df_b_qt], ignore_index=False)

# 5) violin plot으로 그룹 간 분포 비교
my_plot.violinPlot(data=df_qtcheck, y='sum_gamerounds', hue='version', ...)
```

왜 그룹별로 따로 이상치 기준을 잡나? — A/B 그룹의 분포가 다를 수 있어서, 전체 기준(global bound)이 아니라 **그룹별 IQR**로 제거해야 그룹 간 비교가 왜곡되지 않는다. (하드코딩 X, 데이터 기반 동적 기준이라는 원칙.)

7-5. Heatmap — 두 범주 조합에 따른 수치 패턴

개념: Heatmap = **피벗 테이블(행×열×값)**을 **색상으로 시각화**한 것. "히트맵을 그린다" = "피벗 테이블을 먼저 만든다"는 뜻 — 이 순서를 반드시 기억.

표준 처리 순서 (flights 예제)

```
# 1) 전처리: 시각화에 방해되는 타입(문자열 월 등)을 숫자로 변환
df['month'] = df['month'].map({"January":1, ..., "December":12})

# 2) 재구조화: index=행, columns=열, values=색상으로 표현할 값
df2 = pivot_table(df, index="year", columns="month", values="passengers")

# 3) 필요시 정렬 (최신 연도가 위로 오게 등)
df3 = df2.sort_index(ascending=False)
```

4) 시각화

```
sb.heatmap(data=df3, annot=True, fmt="0.1f", linewidths=0.5, cmap="GnBu")
```

파라미터	의미
<code>annot=True</code>	셀 안에 실제 수치값 표시
<code>fmt="0.1f"</code>	annot 표시 형식 (소수점 자리수)
<code>linewidths</code>	셀 사이 구분선 두께
<code>cmap</code> / (모듈에선 <code>palette</code>)	컬러맵 (GnBu, PuRd, Blues 등)

⚠ **파라미터명 주의:** `my_plot.heatmap()` 내부는 `linewidths`(복수형)로 통일. seaborn 원본도 `linewidths`가 맞음 — `linewidth`(단수)로 쓰면 에러.

모듈화 버전

```
my_plot.heatmap(df3, palette="PuRd", width=1200, height=600,
                 title="...", xlabel="Month", ylabel="Year")
```

- `data`는 2차원 형태(보통 `pivot_table` 결과)여야 함

재사용 패턴: "두 범주형 축(연도×월, 지역×상품군 등) + 하나의 집계값" 구조만 되면 언제든지 heatmap 후보. `pivot_table(index=, columns=, values=, aggfunc=)`만 알아 끼우면 됨.

7-6. Bar Plot / Count Plot — 범주별 대표값·빈도 비교

개념 차이

구분	Bar Plot	Count Plot
y축 값	<code>estimator</code> 로 계산된 값 (기본: 평균)	항상 개수(빈도)
필요한 컬럼	x(범주) + y(수치)	x(범주)만 있으면 됨
관계	—	Count Plot = <code>estimator=len</code> 으로 고정된 Bar Plot의 특수 버전

Bar Plot 확장 순서

1) 기본: 집단별 평균

```
sb.barplot(data=origin, x="species", y="body_mass_g")
```

2) 방향 전환: x/y를 바꾸면 가로 막대 (+ hue 쓸 땐 `palette` 필수 세트)

```
sb.barplot(data=origin, x="body_mass_g", y="species", palette="Set2",
           hue="species")
```

```
# 3) 집계 방법 변경 + 정렬 순서 지정
sb.barplot(data=origin, x="species", y="body_mass_g",
           estimator=np.max, order=['Gentoo', 'Chinstrap', 'Adelie'])

# 4) 범주 세분화 (hue로 그룹 나누기)
sb.barplot(data=origin, x="species", y="body_mass_g", hue="sex", palette="pastel")
```

- **estimator**: 막대 높이 계산 방식 (`np.mean`, `np.sum`, `np.median`, `np.max` 등)
- **order**: 막대 순서 직접 지정 (리스트)

Count Plot

```
sb.countplot(data=origin, x="species", palette="spring", hue="sex")
```

→ **estimator=len**이 고정된 barplot. "종별 평균"이 아니라 "종별 개체수"처럼 단순 빈도 비교일 때.

모듈화 버전

```
my_plot.barplot(data=origin, x="species", y="body_mass_g", hue="sex",
                estimator=np.mean, palette="Set2", ...)
my_plot.countplot(data=origin, x="species", hue="sex", palette="tab20", ...)
```

7-7. Pie Plot / Donut Plot — 전체 대비 구성비

범주가 많으면(6개 이상) 가독성이 떨어지므로 상위 N개 + 기타로 묶는 걸 고려할 것.

표준 처리 순서 (cafe_sales 예제)

```
# 1) 집계: 범주별 합계 구하고 내림차순 정렬
df1 = origin[['메뉴명', '금액']].groupby('메뉴명').sum()
df1.sort_values('금액', ascending=False, inplace=True)

# 2) 기본 파이
ax.pie(x=df1['금액'], labels=df1.index, autopct='%1f%%')

# 3) 회전/방향
ax.pie(..., startangle=260, counterclock=False) # 시작각도 + 시계방향(기본값)

# 4) 색상 팔레트 직접 생성해서 적용
colors = sb.color_palette("Set2", n_colors=len(df1['금액']))
ax.pie(..., colors=colors)

# 5) 특정 조각 분리(explode) - 리스트 길이는 데이터 개수와 동일해야 함
explode = [0.1, 0, 0, 0, 0]
ax.pie(..., explode=explode)
```

Donut 만들기 = `wedgeprops`만 추가

```
wedgeprops = {'width':0.7, 'edgecolor':'#ffffff', 'linewidth':3}
ax.pie(..., wedgeprops=wedgeprops)
```

→ `width < 1`이면 가운데가 뚫리는 도넛 형태. `edgecolor/linewidth`는 조각 사이 경계선 스타일.

모듈화 버전

```
my_plot.pieplot(x=df1['금액'], labels=df1.index, autopct="%0.1f%",
                palette='pastel') # 파이
my_plot.pieplot(x=df1['금액'], labels=df1.index, palette='pastel',
                donutchart=True, wedge_width=0.7) # 도넛
```

- `donutchart=True`: 도넛 형태로 (내부적으로 `wedgeprops` 조건부 생성)
- `explode`: 특정 조각만 떨어뜨려 강조
- `counterclock=False`: 시계방향으로 그리기 (기본값)

7-8. Stackplot — 누적 막대 / 100% 누적 막대

```
my_plot.stackplot(data=df, x="지역", y="종사자 수", hue="산업",
                  aggfunc=np.sum, orient='h', ratio=False,
                  text=True, text_format="{:.1f}", palette="tab20")
```

- `orient`: 'v'(세로, `bottom=` 사용) / 'h'(가로, `left=` 사용)
- `ratio=True`: 절대값 대신 **비율(%)로 자동 변환**, `text_format` 없으면 "{:.1f}%"로 강제 지정됨
- `aggfunc`: 누적 전 집계 방식 (합계면 `np.sum`, 평균이면 `np.mean`)
- `text_format`: 막대 안 숫자 표시 형식

⚠ **Stacked bar의 한계**: 첫 세그먼트만 0 기준선이고 나머지는 떠 있어서, "*"카테고리를 가로질러 특정 hue의 값 비교**엔 부적합. 이런 질문("Color별로 어느 Cut에서 제일 비싼가")엔 heatmap, grouped bar, lineplot이 더 적합. → "합계 비교"는 stacked bar가 좋고, "개별 항목 간 비교"는 다른 차트가 좋다.

bar vs barh — 세로/가로 막대 파라미터 대응표

같은 "누적"이라도 방향에 따라 파라미터 이름이 다름 (헛갈려서 에러 자주 남):

	세로 막대 (<code>bar, orient='v'</code>)	가로 막대 (<code>barh, orient='h'</code>)
누적 시작점	<code>bottom=</code>	<code>left=</code>
범주 축	x	y
값 축	y	x
<code>ax.text()</code> 좌표	x=범주, y=누적값	x=누적값, y=범주

```
# 세로
ax.bar(df.index, df['A'], label='A')
ax.bar(df.index, df['B'], bottom=df['A'], label='B') # bottom!

# 가로
ax.barh(df.index, df['A'], label='A')
ax.barh(df.index, df['B'], left=df['A'], label='B') # left!
```

주의: `barh`에 `bottom=`을 넣으면 `TypeError: got multiple values for keyword argument 'bottom'` 발생 (`barh` 내부가 `bottom`을 이미 `y`로 자동 매핑하기 때문).

누적값 계산 패턴 (반복문 안에서 자주 틀리는 부분)

n 번째 세그먼트의 텍스트 y (또는 x) 위치 = 그 전까지 쌓인 값의 합 + 현재 값의 절반

```
for i in df.index:
    ax.text(x=i, y=df['A'][i]/2, s=..., ...) # 1번째:
    0 + A/2
    ax.text(x=i, y=df['A'][i] + df['B'][i]/2, s=..., ...) # 2번째:
    A + B/2
    ax.text(x=i, y=df['A'][i] + df['B'][i] + df['C'][i]/2, s=..., ...) # 3번째:
    A+B + C/2
```

컬럼 수가 유동적이면 `.iloc[:, :i].sum(axis=1)`로 일반화:

```
df.iloc[j, :i].sum() + val/2 # i번째 컬럼까지의 누적합 + 현재값 절반
```

흔한 실수: 반복문 안에서 `df['컬럼']`처럼 `[i]` 인덱싱을 빼먹으면 스칼라가 아니라 시리즈 전체가 계산에 섞여 들어감 → 항상 "이번 반복에서 쓰는 값은 스칼라인가?" 체크.

모듈화 함수 만들 때 체크리스트

- ☐ 함수 시그니처 끝에 : 빠뜨리지 않기
- ☐ 색상 hex 코드 자릿수 확인 (`#ffffff` = 6자리)
- ☐ `if text:` 같은 "옵션 실행 블록"이 `if orient=='v':` / `else:` 분기 **안에** 잘못 감혀있지 않은지 확인 → `orient` 상관없이 실행돼야 하는 코드는 분기 **밖에**
- ☐ `legend()`, `show()`처럼 "전체 다 그린 후 한 번만" 호출해야 하는 코드가 `for` 반복문 **안에** 잘못 들어가 있지 않은지 확인
- ☐ `ratio=True` 같은 옵션이 켜졌을 때 축 범위(`set_ylim/set_xlim`)도 같이 맞춰졌는지 확인

8. Scatter Plot & Convex Hull

8-0. 적용 전 체크리스트 — 새 데이터셋 받았을 때

체크	확인할 것	왜 필요한가
① 결측치	x, y 컬럼에 NaN 있는가?	scatter는 NaN 있어도 조용히 점을 안 찍고 넘어가지만, ConvexHull 은 NaN 있으면 에러남
② x, y dtype	둘 다 연속형(숫자)인가?	범주형을 넣으면 그림이 이상해짐 (그땐 boxplot/violin이 맞음)
③ hue dtype	범주형인가?	<code>plot_hull</code> 로직이 <code>data[hue].unique()</code> 로 군집을 나누는 구조라, hue가 연속형이면 값 하나하나가 다 다른 군집 취급됨 → 연속형을 hue로 쓰려면 먼저 구간화 (binning)
④ hue 범주 개수	대략 6~8개 넘는가?	<code>sb.color_palette(palette)[i]</code> 가 팔레트 기본 색상 수를 넘으면 색이 겹치거나 꼬임 → <code>n_colors=len(classes)</code> 를 명시적으로 넘겨야 함 (<code>origin[hue].nunique()</code> 로 먼저 확인)

→ 전체 그림은: **데이터 품질 체크(결측치/타입/이상치)** → **이 문서(시각화)** 순서로 이어짐.

8-1. 개념 — 언제 무엇을 쓰나

질문	예시	차트
두 연속형 변수가 같이 움직이는가?	날개 길이 ↑ → 몸무게 ↑?	scatter: 점들의 우상향/우하향 패턴
관계의 강도/형태는?	선형인가 곡선인가 관계없음인가	scatter: 점들이 직선에 가깝게 모였는지 흩어졌는지
군집(그룹)별로 관계가 다른가?	종별로 부리 길이-두께 관계가 다른가	scatter + hue
군집의 "영역"이 겹치는가, 분리되는가?	setosa vs versicolor 종이 공간적으로 겹치는 구역이 있는가	Convex Hull : 각 군집 최외곽 점들을 이어서 영역을 면으로 보여줌

핵심 차이: scatter는 점 하나하나(개별 관측치)를 보여주고, Convex Hull은 그 점들의 집합이 "차지하는 영역"을 보여줌. Convex Hull은 scatter 위에 얹는 **추가 레이어**지 단독 차트가 아님.

8-2. Scatter 기본 사용법

```
# 패턴 3 - DataFrame + 컬럼명 ✅ 가장 권장
sb.scatterplot(data=origin, x='flipper_length_mm', y='body_mass_g')
```

기본 마커 스타일링 (hue 없을 때)

```
sb.scatterplot(data=origin, x='flipper_length_mm', y='body_mass_g',
               markers="^",           # 마커 모양
               color="red",           # 마커 색상 (hue 없을 때만 의미 있음)
```

```
s=150,          # 마커 크기 (기본값 100)
edgecolor="black", # 마커 테두리 색상
linewidth=2,    # 마커 테두리 두께
alpha=0.7       # 마커 투명도
)
```

파라미터	의미	주의
markers	마커 모양 ("o", "^", "s" 등)	raw seaborn에선 markers(복수형)
s	마커 크기	기본 100
color	마커 색상	hue가 있으면 무시됨
alpha	투명도	점이 많이 겹칠 때(overplotting) 낮춰서 밀도 표현

군집별 분류 (hue 있을 때)

```
sb.scatterplot(data=origin, x='bill_length_mm', y='bill_depth_mm',
               hue="species", palette="tab10",
               markers="o", s=100, edgecolor="black", linewidth=2, alpha=0.7)
```

8-3. Convex Hull — 개념 및 원리

`scipy.spatial.ConvexHull`은 여러 점들 중 **가장 바깥쪽 점들(꼭짓점)만 뽑아서**, 그 점들을 잇는 다각형(볼록 껍질)을 계산 → "이 군집 데이터를 다 감싸는 가장 작은 다각형"을 구하는 것.

필수 전제조건: 다각형을 만들려면 **최소 3개의 점**이 필요 (2개면 선, 1개면 점).

```
if len(df_c) < 3:
    print("데이터 포인트가 3개 미만이므로 ConvexHull을 그릴 수 없습니다.")
```

군집별 반복 적용 시 `if len(df_c) < 3: continue`로 스킵 처리 필수 (안 하면 에러남).

단일 군집 적용 순서 (원리 이해용)

```
# 1) 특정 군집(setosa)만 필터링
df_c = origin.loc[origin['Species'] == 'setosa', ['Sepal.Length', 'Sepal.Width']]

# 2) ConvexHull 계산 → 외곽 점들의 "인덱스"를 반환
hull = ConvexHull(df_c)
out_index = hull.vertices      # 외곽 꼭짓점들의 인덱스 배열

# 3) 다각형을 "닫기" 위해 첫 인덱스를 마지막에 한 번 더 추가
points = np.append(out_index, out_index[0])

# 4) 선(외곽선) + 면(내부 채우기)으로 표시
ax.plot(df_c.iloc[points, 0], df_c.iloc[points, 1],
```

```
linewidth=1, linestyle=":", color=color)
ax.fill(df_c.iloc[points, 0], df_c.iloc[points, 1],
        alpha=0.1, color=color)
```

암기 포인트

- `hull.vertices`는 "꼭짓점 인덱스"만 주지, 선으로 안 이어줌 → `.iloc[points]`로 좌표 뽑아서 직접 `ax.plot/ax.fill`.
- `np.append(out_index, out_index[0])` — 다각형을 닫는 필수 스텝. 빠뜨리면 도형 한쪽이 뚫린 채로 그려짐.
- `plot`(테두리선) + `fill`(내부 색 채우기) 둘 다 해야 "영역"처럼 보임.

모든 군집에 일괄 적용 (함수화)

```
def plot_hull(data, x, y, hue, palette, ax):
    classes = list(data[hue].unique())
    for i, v in enumerate(classes):
        df_c = data.loc[data[hue] == v, [x, y]]
        if len(df_c) < 3:
            continue
        hull = ConvexHull(df_c)
        points = np.append(hull.vertices, hull.vertices[0])
        color = sb.color_palette(palette)[i] # scatter의 hue 색상과 순서 맞춤
        ax.plot(df_c.iloc[points, 0], df_c.iloc[points, 1], linewidth=1,
                linestyle=":", color=color)
        ax.fill(df_c.iloc[points, 0], df_c.iloc[points, 1], alpha=0.1,
                color=color)
```

일반화 패턴: "군집별로 반복해서 뭔가 그리는" 로직은 `stackplot`의 `for i, col in enumerate(df.columns)`, `kdeplot`의 `for i, cat in enumerate(categories)`와 완전히 같은 구조. `enumerate`로 인덱스+값을 같이 받아 그 인덱스로 팔레트 색상을 뽑는 패턴은 hue 기반 시각화 전체를 관통하는 공식.

8-4. `my_plot.scatterplot()` — outline 옵션 통합 버전

```
def scatterplot(data, x, y, hue=None, marker='o', color=None, size=100,
                edgecolor='ffffff', linewidth=1.5, alpha=1, palette=None,
                outline=True, title=None, xlabel=None, ylabel=None,
                width=1280, height=640, save_path=None):
```

```
# hue/color/palette 스위칭
if hue == None:
    if color is None and palette is not None:
        color = sb.color_palette(palette)[0]
        palette = None
else:
    color = None
```

```
# ... scatterplot 그리기 ...

# 외곽선 그리기 - hue가 있을 때만 의미 있음
if outline and hue is not None:
    plot_hull(data=data, x=x, y=y, hue=hue, palette=palette, ax=ax)
```

`outline=True`가 기본값이지만, `hue`가 없으면 자동으로 무시됨 (`hue is not None` 조건) — "군집이 없는데 외곽선을 어떻게 나눠 그리겠어"라는 전제가 조건문으로 명시된 것.

사용 예시

```
# hue 있으면 outline 기본 True로 자동 그려짐
my_plot.scatterplot(data=origin, x='Petal.Length', y='Petal.Width', hue='Species',
                    size=100, edgecolor="#000000", linewidth=1.5, palette='Set2')

# outline 끄고 싶으면 명시적으로 False
my_plot.scatterplot(data=origin, x='Petal.Length', y='Petal.Width', hue='Species',
                    palette='Set2', outline=False)
```

8-5. 분석 진행 순서

1. 패키지 참조 (helpers, seaborn, numpy, scipy.spatial.ConvexHull)
2. 데이터셋 로드
3. raw seaborn으로 기본 산점도 먼저 그려서 군집 분포 확인
4. 군집 하나만 떼어내서 ConvexHull 원리 손으로 확인
 - 3개 미만 예외처리 → `hull.vertices` → 다각형 달기 → `plot+fill`
5. 위 로직을 모든 군집에 반복 적용하는 함수(`plot_hull`)로 일반화
6. 산점도 + `plot_hull`를 합쳐서 최종 시각화
7. 모듈화 함수(`my_plot.scatterplot`)에 `outline` 옵션으로 통합 확인
8. 연습문제로 다른 데이터셋에 그대로 적용 (컬럼명만 바꿔서 바로 동작해야 정상)

→ "raw로 원리 확인 → 함수로 일반화 → 모듈에 통합 → 새 데이터로 재검증" 이 4단계는 LAB06 전체에서 반복되는 패턴.

9. LmPlot & PairPlot (figure-level 함수)

9-1. Implot이란

`sb.lmplot(data, x, y, ...)` : 산점도(scatter) + 선형회귀선(regression line)을 같이 그려주는 함수.

- **figure-level 함수**다 → `plt.subplots()`로 만든 `ax`에 그리는 게 아니라 자기가 알아서 새 Figure를 만들. → 사이즈 지정 방식이 `width/height`가 아니라 `height`(그래프 1개 높이) + `aspect`(가로/세로 비율)로 지정됨.
- 결과로 `g`라는 **FacetGrid** 객체가 반환됨:
 - `g.fig` → figure 객체 (dpi, subtitle 설정할 때 씀)
 - `g.axes.flatten()` → 내부 `ax`들을 1차원 리스트로 (col/row로 나뉘면 `ax`가 여러 개)

9-2. pairplot이란

`sb.pairplot(data, ...)` : 데이터의 모든 수치형 변수 조합을 자동으로 산점도 매트릭스로 그림.

- 대각선(자기 자신과의 조합)은 분포 그래프(`diag_kind='hist'` 또는 `'kde'`)로 채워짐.
- 역시 figure-level 함수 → `g.fig, g.axes.flatten()` 패턴 동일 적용.
- `x_vars, y_vars` 지정 시 전체 조합이 아니라 원하는 변수만 선택 가능 (변수 많을 때 유용).

9-3. scatter_kws / line_kws 구조 (lplot)

```
sb.lplot(...,
    scatter_kws={"edgecolor": "...", "s": 50, "alpha": 0.8},
    line_kws={"color": "...", "linestyle": "--", "linewidth": 2}
)
```

- 점 스타일은 `scatter_kws`, 회귀선 스타일은 `line_kws`에 덕셔너리로 몰아넣는 구조.
- `hue`로 범주를 나누면 팔레트가 색을 자동 배정하니까 `scatter_kws/line_kws` 안에 `color`를 따로 지정하면 안 됨 (충돌).

9-4. pairplot에서 회귀선 켤 때 구조가 바뀌는 것 (헷갈리기 쉬움)

- 기본 `pairplot(kind='scatter', 기본값)`: `plot_kws={"s":30, "alpha":0.7}` 처럼 평평하게 씬.
- `kind='reg'`로 회귀선을 켜는 순간, seaborn 내부적으로 `lplot`과 같은 로직을 타서 `plot_kws` 안에 다시 `scatter_kws, line_kws`를 중첩해야 함:

```
plot_kws={
    'scatter_kws': {'s':15, 'alpha':0.5},
    'line_kws': {'linewidth':2, 'linestyle':'--'}
}
```

→ `***kind이 바뀌면 옵션을 넣는 계층 구조도 바뀐다***`는 걸 기억할 것.

9-5. lplot 상세 문법

```
g = sb.lplot(
    data=origin, x='bill_length_mm', y='bill_depth_mm',
    height=height,          # 그래프 1개의 세로 높이(inch)
    aspect=width/height,    # 가로세로 비율 (aspect * height = 가로 크기)
    hue='species',          # 범주별 색상 구분
    col='sex',              # 열 방향으로 패킷 분할 (row도 동일 원리)
    legend=False,          # 자동 범례 끄기 (겹치는 문제 있어서 수동 위치 지정 권
장)
    markers=["s","o","D"],  # 마커 모양 리스트 - 개수는 hue 범주 수와 반드시 일치
    palette="Set2",
    scatter_kws={"edgecolor":"#000000","linewidths":1,"s":50,"alpha":0.8},
    line_kws={"linestyle":"--","linewidth":2}
)
```

```

g.fig.set_dpi(200)
g.fig.set_tight_layout(True)
g.fig.suptitle('제목', fontsize=24, fontweight=500, y=1)

ax = g.axes.flatten()          # col/row로 나뉘면 ax가 여러개 → 리스트로 순회
for a in ax:
    a.set_xlabel('...'); a.set_ylabel('...')
    a.legend(bbox_to_anchor=(1,1), loc='upper left')
    a.grid(True, alpha=0.5)

my_plot.show()

```

주의할 점

- hue 쓸 때 `scatter_kws/line_kws`에 색상(color) 키를 넣으면 palette와 충돌 → 빼야 함.
- `markers` 리스트 길이는 반드시 hue 범주 개수와 같아야 함 (안 그러면 ValueError).
- `col/row`로 나누면 `ax`가 배열이 되므로 `ax[0]` 하나만 만지지 말고 `for` 반복문으로 전체 처리.

9-6. pairplot 상세 문법

```

g = sb.pairplot(
    origin,
    hue='species', markers=['o', 's', '^'], palette='cool',
    diag_kind='kde',          # 대각선: 'hist' 또는 'kde'
    kind='reg',               # 'scatter'(기본) 또는 'reg'(회귀선)
    plot_kws={                # kind='reg'일 때만 이 중첩 구조 사용
        'scatter_kws': {'s':15, 'alpha':0.5},
        'line_kws': {'linewidth':2, 'linestyle':'--'}
    },
    x_vars=["bill_length_mm", "bill_depth_mm"], # 선택적으로 일부 변수만
    y_vars=["bill_length_mm", "bill_depth_mm"]
)
g.fig.set_dpi(200)
g.fig.set_figwidth(...); g.fig.set_figheight(...)
g.fig.suptitle('제목', fontsize=16, fontweight='bold')

for ax in g.axes.flatten():
    ax.set_axisbelow(True)
    ax.grid(True, alpha=0.5)

```

주의할 점

- `kind='scatter'`(기본)일 땐 `plot_kws`가 평평한 딕셔너리, `kind='reg'`일 땐 `scatter_kws/line_kws`로 중첩.
- 변수가 너무 많으면 매트릭스가 커져 느려짐 → `x_vars/y_vars`로 관심 변수만 추리는 습관.

9-7. `my_plot.lmplot()` / `my_plot.pairplot()` 사용법

```
# lmpplot
my_plot.lmpplot(
    data=origin, x='bill_length_mm', y='bill_depth_mm',
    title='...', xlabel='...', ylabel='...',
    hue='species', palette="Set2", col="sex",
    markers=["s", "o", "D"],
    scatter_edgecolor="#000000", scatter_linewidths=1, scatter_size=50,
    linestyle="--", linecolor="#0066ff", linewidth=2
)

# pairplot
my_plot.pairplot(
    data=origin, hue='species', reg=True, diag_kind='kde',
    title="...", width=1600, height=1200,
    markers=['o', 's', '^'], palette='Set2',
    scatter_size=5, scatter_alpha=0.7,
    linecolor='#0066ff', linewidth=1, linestyle='--'
)
```

- 호출부는 raw seaborn 버전보다 훨씬 짧고, "무엇을 그릴지"에만 집중하면 됨.
- 내부적으로 dpi 설정 → subtitle → grid 반복 적용 → show()를 함수가 대신 다 해줌.

9-8. 알아둘 이슈

- markers 인자에 문자열 하나("^")를 줄 때와 리스트(["s", "o", "D"])를 줄 때가 다름: hue 없을 땐 문자열 하나, hue 있을 땐 범주 수만큼 리스트로.
- lmpplot/pairplot은 다른 함수들과 다르게 ax=None 파라미터가 없음 — ax를 넘기려고 하면 안 됨.

9-9. 다른 데이터셋에 적용할 때 체크리스트

1. x, y로 쓸 컬럼이 수치형인지 확인 (origin.dtypes).
2. hue로 쓸 컬럼의 범주 개수를 먼저 세어봄 (origin['컬럼'].nunique()) → markers 리스트 길이를 여기 맞춤.
3. 결측치가 있으면 seaborn이 알아서 빼고 그리긴 하지만, 회귀선(reg=True) 쓸 때는 결측 많으면 왜곡될 수 있으니 먼저 isnull().sum() 확인.
4. 컬럼 수가 많으면 pairplot은 무조건 x_vars/y_vars로 관심 변수만 골라서 그리기.
5. 축 라벨은 한글로 바꿔서 넘기기 — 컬럼명이 영어/코드성이면 사람이 읽을 이름으로.
6. raw seaborn으로 먼저 1~2번 확인 후, 확실해지면 my_plot 함수 호출로 전환.

9-10. 모듈을 확장할 때 사고 순서 (새 그래프 타입 추가 시)

1. 이게 axes-level인지 figure-level인지 먼저 확인 (seaborn 공식 문서 명시) → 4-3 패턴을 쓰지, lmpplot/pairplot 예외 패턴을 쓰지 결정.
2. raw seaborn으로 원하는 결과가 나올 때까지 직접 코딩으로 먼저 실험.
3. "이 옵션은 항상 이 값으로 고정해도 되겠다" 싶은 건 기본값으로, "매번 바뀔 것 같다" 싶은 건 파라미터로 노출.
4. 중첩 딕셔너리가 있으면 평평하게 풀어서 파라미터명 짓기 (scatter_size, scatter_alpha).
5. init()/show() 재사용 가능하면 그대로, 안 되면 (figure-level) 이유를 주석으로 남김.
6. 함수 다 짜면 최소 2개 이상의 다른 데이터셋으로 테스트해서 특정 데이터에만 맞춰진 하드코딩이 없는지 확인.

7. docstring(Args 설명)까지 채워 넣기.

10. SubPlot (서브플롯)

10-1. 개념

서브플롯 = 하나의 도화지(fig) 안에 여러 개의 작은 그래프(ax)를 나눠서 그리는 것. (2장 참고)

```
fig, ax = plt.subplots(rows, cols, figsize=..., dpi=...)
```

- `rows=1, cols=1`이면 `ax`는 단일 Axes 객체
- `rows>1` 또는 `cols>1`이면 `ax`는 2차원 배열(예: 2행 3열 → `shape=(2,3)`)

```
ax = ax.flatten()
```

- 2차원 배열은 `ax[0][1]` 식으로 접근해야 해서 반복문 돌리기 불편 → `flatten()`으로 1차원으로 펴서 `ax[0]`, `ax[1]`... 순서 접근.
- **인덱스 순서는 행 우선(row-major)**: 2행 3열이면 `ax[0]~ax[2]`가 1행, `ax[3]~ax[5]`가 2행.
- `len(ax)`로 몇 개 만들어졌는지 항상 체크하는 습관.

10-2. raw matplotlib vs my_plot 모듈 비교

방식 A — 직접 구현 (raw)

```
fig, ax = plt.subplots(rows, figsize=figsize, dpi=my_dpi)
ax = ax.flatten()
fig.suptitle('제목', fontsize=32, fontweight=500)
plt.tight_layout()
plt.show()
plt.close()
```

→ 매번 이 5~6줄을 반복. 그래프마다 스타일을 손으로 맞춰야 함.

방식 B — my_plot 모듈화

```
fig, ax = my_plot.init(width=600, height=400, rows=2, cols=3, title='...')
my_plot.show()
```

→ 같은 결과를 함수 호출 2줄로 끝냄. 스타일이 항상 일관됨.

10-3. my_plot이 서브플롯에서도 동일 패턴을 유지하는 이유

`lineplot`, `histplot`, `boxplot`, `scatterplot` 등 전부 4-3에서 본 `ax=None` 패턴을 그대로 사용:


```
def 어떤plot(data=None, x=None, y=None, ..., ax=None):
    fig = None
    if ax is None:
        fig, ax = init(width=width, height=height, title=title, ...)

    sb.어떤plot(data=data, x=x, y=y, ..., ax=ax)  # 실제 그리기

    if fig is not None:
        show(save_path=save_path)
```

상 황	ax를 안 넘겨줌 (ax=None)	ax를 넘겨줌 (서브플롯의 한 칸)
동 작	함수가 직접 fig/ax를 새로 만들	넘겨받은 ax에 그냥 그림
결 과	그래프 하나만 단독 출력 (show() 실행)	show() 실행 안 함 (전체 다 그린 후 한 번만)
예 시	my_plot.lineplot(data=df, x='년도', y='발생건수')	my_plot.lineplot(data=df, x='년도', y='발 생건수', ax=ax[0])

10-4. 서브플롯 그릴 때 진행 순서 (실전 흐름)

처음부터 서브플롯 6~12개를 한 번에 다 그리려고 하지 말고, 아래처럼 단계별 검증:

```
[1] 데이터 불러오기
    ↓
[2] 전처리 - groupby, drop 등으로 분석 단위 맞추기
    ↓
[3] 그래프별로 필요한 데이터 "따로" 준비
    - 히트맵용: pivot_table로 재배치
    - 파이차트용: groupby + sum + reset_index로 집계
    ↓
[4] 서브플롯 틀(fig, ax) 초기화 - 몇 행 몇 열이 필요한지 먼저 결정
    ↓
[5] ax 하나하나에 그래프 얹기 (인덱스 순서대로)
    ↓
[6] show() 한 번으로 전체 출력
```

왜 [3]에서 데이터를 미리 다 따로 준비해두는가? — 그래프 종류마다 요구하는 데이터 모양이 다름(heatmap은 pivot, pie는 groupby+sum, box/scatter는 원본 그대로) → 그리는 도중 즉석 처리하면 코드가 지저분해지고 에러 나기 쉬움.

점진적 검증 단계

- **Step 1:** 서브플롯 틀만 먼저 만들어서 개수 확인

```
fig, ax = my_plot.init(rows=2, cols=3, width=800, height=450, title='제목')
print(f'총 {len(ax)}개의 ax객체가 생성되었습니다.')
my_plot.show()
```

- **Step 2:** ax 하나에만 그래프 하나 넣어서 테스트 (나머지는 비워둔 채, 파라미터 오타·컬럼명 오류를 여기서 잡는 게 전체 채운 뒤 디버깅보다 빠름)
- **Step 3:** ax[0] 확인 → ax[1] 추가 → ax[2] 추가... 하나씩 늘려가며 채우기 (특히 이전에 버그 났던 함수는 단독 실행 확인 후 서브플롯에 넣는 게 안전)
- **Step 4:** 전체를 한 번에 실행해서 최종 확인

왜 이렇게 하는가: 서브플롯은 그래프 하나하나가 각각 데이터/파라미터가 다르기 때문에, 한 곳에서 에러 나면 어디가 문제인지 찾기 어려움. 하나씩 검증하고 쌓아 올리면 "방금 추가한 것"만 의심하면 되므로 디버깅 범위가 좁아짐.

10-5. 다른 데이터셋에 적용할 때 체크리스트

- ☐ origin.head()로 컬럼명/타입 먼저 확인
- ☐ 그리려는 그래프 종류별로 **필요한 데이터 형태** 파악 (원본 그대로 되는지 / groupby / pivot 필요한지)
- ☐ 몇 개의 그래프를 그릴지 정하고 → rows * cols >= 그래프 개수로 배치 먼저 계산
- ☐ my_plot.init()으로 빈 틀만 먼저 만들어서 len(ax) 확인
- ☐ 그래프 하나씩 넣어보며 ax=ax[i] 인덱스가 순서대로 맞는지 확인 (꼬이면 엉뚱한 칸에 들어감)
- ☐ hue/palette 쓰는 함수는 범례가 겹치지 않는지 확인 (bbox_to_anchor 옵션 조정)
- ☐ 마지막에 전체 실행해서 supitle, 여백(tight_layout)이 깨지지 않는지 확인

10-6. 서브플롯용 그래프 함수 목록 (용도 빠른 참조)

함수	용도	필요한 데이터 형태
lineplot	시계열 추이	원본 그대로 (x=시간축)
kdeplot	분포의 밀도 곡선	원본 그대로 (수치형 컬럼)
histplot	구간별 빈도	원본 그대로 (수치형 컬럼)
boxplot / violinPlot	분포 + 이상치 확인	원본 그대로
heatmap	두 범주 교차 패턴	pivot_table로 재배치 필요
barplot / countplot	범주별 집계/개수 비교	원본 그대로 (estimator로 집계방식 지정)
pieplot	비율 확인	groupby+sum으로 집계 필요
stackplot	누적 비교	함수 내부에서 자동으로 pivot_table 수행
scatterplot	두 변수 간 관계	원본 그대로 (hue 쓰면 plot_hull로 외곽선까지)
lmlot	관계 + 회귀선	원본 그대로 (단, 서브플롯 ax 미지원 — figure-level)
pairplot	여러 변수 조합 한 번에	원본 그대로 (역시 ax 미지원, figure-level)

⚠ lmlot, pairplot은 figure-level 함수라서 서브플롯의 한 칸에 못 들어감. 서브플롯에 넣고 싶으면 별도 처리(이미지로 저장 후 삽입 등) 필요 — 현재 구조에서는 지원 안 됨.

11. Twin Axis (이중 축)

11-1. 왜 필요한가

"발생건수"랑 "부상자수"를 같은 그래프 안에, 겹쳐서, 같이 보고 싶다. 문제는 이 둘의 값의 스케일이 완전히 다를 수 있음 (수천 단위 vs 수백 단위) → 같은 y축에 그리면 한쪽 선이 거의 0에 붙어 안 보임.

해결책: x축(년도)은 공유하되, y축을 왼쪽/오른쪽에 각각 따로 두는 것 = **이중 축(twin axis)**.



11-2. 서브플롯과의 차이

	서브플롯 (Sub Plot)	이중 축 (Twin Axis)
그래프 개수	여러 개를 나눠서 따로따로	한 화면에 겹쳐서 하나로
목적	서로 다른 그래프 종류를 한눈에 모아보기	스케일이 다른 두 변수를 같은 시점끼리 비교
x축	각 ax마다 따로 있음	공유함
y축	각 ax마다 따로 있음	왼쪽/오른쪽으로 나눠서 각자

한 문장으로: 서브플롯은 "따로 보여주는 것", 이중 축은 "겹쳐서 비교하는 것".

11-3. `ax.twinx()` 뜯어보기

```
fig, ax = my_plot.init()           # ① 기본 도화지 + 왼쪽 축(ax) 하나 생성
ax_twinx = ax.twinx()              # ② ax와 x축을 공유하는 '오른쪽 축' 하나 더 생성
ax = (ax, ax_twinx)                # ③ 둘을 튜플로 묶어서 관리
```

`ax.twinx()`: "이 ax랑 x축은 똑같이 쓰되, y축만 새로 하나 더 만들어서 오른쪽에 붙여줘."

왜 튜플로 묶어서 관리하나 — 서브플롯의 `ax[0]`, `ax[1]` 인덱스 접근 패턴을 그대로 이어가려는 것:

```
ax = (ax, ax_twinx)
my_plot.lineplot(..., ax=ax[0])  # 왼쪽 축에 그리기
my_plot.lineplot(..., ax=ax[1])  # 오른쪽 축에 그리기
```

→ 서브플롯이든 이중 축이든 "여러 개의 ax를 인덱스로 꺼내 쓴다"는 사용법이 동일하게 유지되도록 설계.

11-4. `my_plot.init(twinx=True)`

```
fig, ax = my_plot.init(title='멀티 축을 갖는 그래프', twinx=True)
```

위 3줄(`init()` → `twinx()` → 튜플 묶기)을 전부 내부에서 처리해줌. 결과적으로 `ax`는 바로 (왼쪽축, 오른쪽축) 튜플로 나옴.

내부적으로 개념상 이렇게 동작:

```
def init(..., twinx=False):
    fig, ax = plt.subplots(...) # 기본 축 생성
    ...
    if twinx:
        ax_right = ax.twinx() # 오른쪽 축 추가 생성
        ax = (ax, ax_right) # 튜플로 묶어서 반환
    return fig, ax
```

모듈 확장의 좋은 예시: 자주 반복되는 3줄짜리 패턴을 함수 파라미터 하나(`twinx=True`)로 흡수시킨 것. "이 패턴, 매번 반복되네?" 싶으면 옵션 파라미터로 빼는 식으로 사고하면 됨.

11-5. 실전 코드 흐름

```
# 1) 데이터 준비
origin = load_data('traffic_acc')
df = origin.drop('월', axis=1).groupby('년도').mean()

# 2) 이중 축 캔버스 초기화 - 이 한 줄이 왼쪽/오른쪽 축을 다 만들어줌
fig, ax = my_plot.init(title='교통사고 발생건수, 부상자수 추이', twinx=True)

# 3) 왼쪽 축(ax[0])에 발생건수 그리기
my_plot.lineplot(data=df, x=df.index, y='발생건수', ylabel='발생건수',
                  color='tab:blue', ax=ax[0])

# 4) 오른쪽 축(ax[1])에 부상자수 그리기 - 같은 x축(년도) 위에 겹쳐짐
my_plot.lineplot(data=df, x=df.index, y='부상자수', ylabel='부상자수',
                  color='tab:orange', ax=ax[1])

# 5) 출력
my_plot.show()
```

`color='tab:blue'` / `color='tab:orange'`처럼 다르게 지정한 게 우연이 아님. 두 선이 어느 y축(왼쪽/오른쪽)에 속하는지 색으로 구분해줘야 헷갈리지 않음. 이중 축 그래프에서 색상 구분은 사실상 필수.

11-6. 이중 축 사용 시 주의할 점

- 서로 다른 스케일일 때만 쓰는 게 맞음. 값 범위가 비슷하면 그냥 같은 y축에 같이 그리는 게 더 직관적 (이중 축은 두 선의 "실제 교차점"이 착시를 일으킬 수 있음 — 스케일이 다르므로 교차점이 실제로 의미 있는 게 아닐 수 있음).
- 범례(legend)가 자동으로 합쳐지지 않음. `ax[0]`, `ax[1]`은 서로 다른 축 객체라서 각자 `legend()`를 호출하면 따로 뜨거나 겹칠 수 있음 → 필요하면 두 축의 handle을 합쳐 하나의 범례로 만드는 별도 처리 필요.
- 색상으로 반드시 축을 구분할 것.
- `twinx`는 "x축 공유 + y축 분리"임. 반대로 y축 공유 + x축 분리는 `twiny()` (개념은 대칭).

11-7. 서브플롯 vs 이중 축 — 선택 기준

1. "여러 변수/그래프를 각각 독립적으로 보고 싶다" → 서브플롯 (`my_plot.init(rows=, cols=)`)
2. "딱 두 변수를 같은 시점(x축) 기준으로 겹쳐서 비교하고 싶다. 근데 스케일이 다르다" → 이중 축 (`my_plot.init(twinx=True)`)
3. "스케일도 비슷하고 그냥 겹쳐 보기만 하면 된다" → 이중 축도 필요 없이 하나의 ax에 `lineplot`을 두 번 그리면 됨 (같은 y축 공유)

12. `my_plot` 모듈 함정 노트 (버그성 주의사항) — 원본 코드 분석에서 발견

12-1. `ax=`를 넘기면 `title`, `xlabel`, `ylabel`, `width`, `height`가 조용히 무시됨

`lineplot` 원본 코드 구조:

```
def lineplot(data=None, x=None, y=None, hue=None,
             title=None, xlabel=None, ylabel=None,
             ..., width=1280, height=540, save_path=None, ax=None):

    fig = None
    if ax is None:
        fig, ax = init(width=width, height=height, title=title,
                      xlabel=xlabel, ylabel=ylabel)    # ← title/xlabel/ylabel이
여기서만 적용됨

    sb.lineplot(data=data, x=x, y=y, hue=hue, ..., ax=ax) # ← 여기서
title/xlabel/ylabel을 아예 안 씀

    if fig is not None:
        show(save_path=save_path)
```

핵심: `title`, `xlabel`, `ylabel`, `width`, `height`는 오직 `init()`을 새로 호출할 때만 적용됨. `init()`은 `ax is None` 일 때만 호출됨. 즉:

상황	<code>title/xlabel/ylabel</code> 파라미터를 줬을 때
<code>ax=None</code> (함수가 직접 <code>fig/ax</code> 생성)	정상 적용됨
<code>ax=ax[0]</code> 처럼 넘겨받은 <code>ax</code> 사용	완전히 무시됨. <code>sb.lineplot()</code> 호출 시 이 값들을 아예 안 씀

실제 사례 (`twinx` 코드에서 이미 이 문제가 숨어있었음)

```
my_plot.lineplot(data=df, x=df.index, y='발생건수', ylabel='발생건수',
                  color='tab:blue', ax=ax[0])
```

`ylabel='발생건수'`를 줬는데 `ax=ax[0]`도 같이 줬기 때문에 이 **ylabel**은 실제로 왼쪽 축에 절대 안 나타남.

이게 왜 위험하냐면 — **에러가 안 남**. 코드가 조용히 실행되고 그래프도 정상적으로 그려짐. 다만 y축 레이블만 안 붙는 거라서, 한참 뒤에 그래프 보다가 "어? 왜 라벨이 없지?" 하고 나서야 알아차리게 됨. 이런 종류의 버그 (조용히 무시되는 파라미터)가 제일 잡기 어려움.

대응법: `ax=`를 직접 넘기는 상황(서브플롯/twinx)에서 축 제목이 필요하다면, **함수 호출 후 직접 ax 객체에 설정**해야 함:

```
my_plot.lineplot(data=df, x=df.index, y='발생건수', color='tab:blue', ax=ax[0])
ax[0].set_ylabel('발생건수') # ← 이렇게 직접 지정해야 실제로 반영됨

my_plot.lineplot(data=df, x=df.index, y='부상자수', color='tab:orange', ax=ax[1])
ax[1].set_ylabel('부상자수') # ← 마찬가지로
```

또는 근본적 해결책(모듈을 직접 고칠 수 있는 상황이면): `ax`가 새로 만들어졌든 넘겨받았든 항상 라벨을 적용하도록 함수 내부를 바꾸는 것:

```
sb.lineplot(..., ax=ax)
if ylabel: ax.set_ylabel(ylabel) # ax 유무와 상관없이 항상 적용
if xlabel: ax.set_xlabel(xlabel)
if title: ax.set_title(title)
```

여기서 배울 사고방식: "이 파라미터가 **모든 경로(ax 있을 때/없을 때)**에서 다 적용되는가?"를 함수 볼 때마다 다 체크하는 습관. `histplot`, `boxplot`, `barplot` 등 다른 함수들도 전부 같은 구조라서, `ax=`를 직접 넘기는 모든 상황에서 `title/xlabel/ylabel`은 항상 무시된다는 걸 기본값으로 기억.

12-2. `twinx=True`는 `rows=1, cols=1`일 때만 작동함 (구조적 제약)

`init()` 원본:

```
def init(width=1280, height=640, rows=1, cols=1, ..., twinx=False):
    fig, ax = plt.subplots(rows, cols, figsize=my_figsize, dpi=200)

    if rows > 1 or cols > 1:
        ax = ax.flatten() # ← 이 경우 ax는 배열(array)이 됨
        ...
    else:
        ... # ← 이 경우에만 ax가 단일 Axes 객체

    if twinx:
        ax_right = ax.twinx() # ← array는 .twinx() 메서드가 없음!
        ax = (ax, ax_right)
```

```
return fig, ax
```

.twinx()는 단일 Axes 객체에만 있는 메서드지, 배열(array)에는 없음. 그래서:

- `rows=1, cols=1`(기본값) + `twinx=True` → 정상 작동
- `rows=2, cols=3` + `twinx=True`처럼 서브플롯 + 이중 축을 동시에 시도하면 → 에러남 (`AttributeError: 'numpy.ndarray' object has no attribute 'twinx'`)

→ 지금 구조로는 "여러 칸짜리 서브플롯 각각에 이중 축을 넣는 것"은 지원 안 됨. 필요해지면 직접 `plt.subplots()`로 만든 뒤 각 ax마다 개별적으로 `.twinx()`를 호출하는 방식으로 짜야 함.

12-3. 최종 요약 문장 모음

- `fig` = 도화지, `ax` = 그 안의 그림 영역. 여러 `ax`로 나누면 서브플롯, x축만 공유하고 y축을 나누면 이중 축.
- 서브플롯은 "따로 보기", 이중 축은 "겹쳐서 비교하기".
- 모듈(`my_plot`)은 반복되는 matplotlib 패턴을 함수 옵션 하나로 흡수한 것. 새로운 패턴이 반복되면 "이거 옵션으로 뺄 수 있지 않을까?" 사고하는 게 모듈 확장의 핵심.
- axes-level 함수(`ax=` 받는 것)만 서브플롯/이중 축 둘 다에 넣을 수 있고, figure-level 함수(`Implot`, `pairplot`)는 둘 다 못 들어감.
- `ax=`를 직접 넘길 때는 `title/xlabel/ylabel/width/height`가 조용히 무시된다는 것 — 항상 의심할 것.
- `twinx=True`는 단일 Axes(`rows=1, cols=1`)에서만 작동.

13. EDA 파이프라인 — `my_qtcheck` 모듈

시각화 "전에" 데이터 품질을 점검하는 헬퍼 모듈. 전체 EDA→시각화 파이프라인 순서:

1. 데이터 로드 (`load_data`)
2. 타입/구조 점검 (`my_qtcheck.set_type`, `numerical_summary`, `categorical_summary`)
3. 시각화 목적에 맞는 전처리 (`pivot_table`, `melt`, `groupby`)
4. 그래프 함수 호출 (`sb.xxxplot` 원본 → `my_plot.xxxplot` 모듈화)
5. 해석

중요 습관: 그래프 그리기 "전에" 반드시 데이터 구조를 한 번 짚어봐야 함 (`df.head()`, `df.info()`, `pivot_table` 결과 확인). 복불만 하고 변수명/타이틀을 안 바꾸는 실수가 반복되니, "이 그래프가 지금 어떤 df/컬럼을 보고 있는지" 매번 확인하는 습관으로 고칠 것.

13-1. `my_qtcheck.py` 함수 목록

함수	역할	언제 쓰나
<code>set_type()</code>	컬럼 타입 일괄 변환 (int/float/string/category/datetime) + <code>df.info()</code> 자동 출력	데이터 로드 직후, 분석 시작 전 항상 1회
<code>get_number_column_names()</code>	숫자형 컬럼명 리스트 반환	다른 summary 함수 내부에서도

함수	역할	언제 쓰나
		사용
<code>get_categorical_column_names()</code>	범주형(category) 컬럼명 리스트 반환	다른 summary 함수 내부에서도 사용
<code>check_duplicates()</code>	중복행 탐지 + (옵션) 제거	로드 직후
<code>check_missing_values()</code>	컬럼별 결측치 개수·비율 표 반환	로드 직후
<code>categorical_summary()</code>	범주형 기술통계 + <code>value_counts()</code> 요약, Excel 저장 옵션	범주형 변수 분포 파악
<code>numerical_summary()</code>	핵심 함수 — 숫자형 컬럼 전체 진단 (아래 상세)	이상치/분포 파악 후 시각화 방향 결 정

⚠ 주의: docstring에 "반드시 `set_type`으로 범주형 처리 후 사용"이라고 명시됨 — 범주형으로 남아있어야 할 컬럼(코딩 문자 등)을 숫자형 통계에 섞으면 왜곡됨.

13-2. `numerical_summary()`가 자동으로 뽑아주는 것 (핵심)

1. 평균-==중앙값== 상대 차이율 `rel_diff = |mean - 50%| / 50%`
 - $< 0.1 \rightarrow \text{similar}$, $< 0.5 \rightarrow \text{diff}$, 그 외 $\rightarrow \text{large_diff}$ (분포가 좌우대칭에 가까운지 1차 판단)
2. IQR 기반 이상치 경계
 - `iqr = Q3 - Q1`
 - `upper_bound = Q3 + 1.5*iqr`, `lower_bound = Q1 - 1.5*iqr`
 - 이 경계로 상한/하한/전체 이상치 개수·비율까지 자동 계산
3. 왜도(skew) / 첨도(kurt) 및 해석
 - `skew < -0.5 \rightarrow \text{left tail}`, $> 0.5 \rightarrow \text{right tail}$, 그 외 $\rightarrow \text{symmetric}$
 - `kurt < 0 \rightarrow \text{platykurtic}`(평평), $> 0 \rightarrow \text{leptokurtic}$ (뽕족), $0 \rightarrow \text{mesokurtic}$
4. 로그 변환 필요성 자동 판정 (`log_need` 컬럼)
 - `skew  $\geq 1$  또는 (skew > 0.5 and kurt > 0)  $\rightarrow \text{log1p}$  추천`
 - `skew  $\leq -1$  또는 (skew < -0.5 and kurt > 0)  $\rightarrow \text{reverse\_log1p}$  추천`
 - 그 외 $\rightarrow \text{none}$

재사용 포인트: `numerical_summary()`를 돌리면 "이 컬럼은 이상치가 몇 % 있고, 오른쪽 꼬리가 길고, 로그 변환이 필요하다"는 걸 표 하나로 한 번에 알 수 있음. 즉, `boxplot/histplot/kdeplot` 중 뭘 그려야 할지 이 표가 먼저 알려줌 $\rightarrow$ 시각화 전에 항상 `numerical_summary()`부터 돌리는 습관.

13-3. `my_plot.py` 공통 구조 (다시 정리)

```
def init(width=1280, height=640, rows=1, cols=1, title=None, xlabel=None,
         ylabel=None, grid=True):
    # figsize, title/label, grid까지 세팅한 fig, ax 반환
def show(save_path=None):
    # tight_layout  $\rightarrow$  show  $\rightarrow$  close (save_path 있으면 저장까지)
```


lineplot, kdeplot, histplot, boxplot, violinPlot, heatmap, barplot, countplot, pieplot 전부 "init → seaborn 호출 → show" 골격이 동일 → 새 그래프 함수 추가할 때도 이 골격만 따라가면 됨.

13-4. Olist(실전 프로젝트) 적용 연결 지점

- `numerical_summary()`의 동적 percentile/IQR 계산 로직 → SLA 임계값(delay_p80, handling_p80 등)처럼 하드코딩 대신 데이터 기반 산출에 재사용 가능.
- **Heatmap**은 "지역×월" 매출/주문 패턴, "카테고리×배송등급" 조합 등 다차원 교차 분석에 바로 적용 가능 (pivot_table만 갈아 끼우면 됨).
- **Bar/Count Plot + melt()** 조합은 "카테고리별 반복구매 비율"처럼 여러 지표를 하나의 그래프에 비교할 때 활용.
- **Violin Plot**은 SLA 등급별 배송 소요일 분포처럼, 단순 평균 비교(barplot)로는 안 보이는 "분포 모양 차이"를 보여줘야 할 때 유용.

14. 새 데이터셋 적용 시 종합 체크리스트

앞으로 다른 데이터셋으로 그래프 그릴 때 이 순서로:

1. **환경 세팅(1장)** 이 세션에서 이미 했는지 확인 (폰트/rcParams는 세션당 한 번이면 충분).
2. `origin.head()`, `df.info()`로 컬럼명/타입/결측 먼저 확인.
3. `my_qtcheck.set_type()` → `check_duplicates()` → `check_missing_values()` → `numerical_summary()` / `categorical_summary()` 순으로 EDA 진단 (13장).
4. 그리려는 그래프가 **axes-level**인지 **figure-level**인지 먼저 구분 (2-7 참고)
 - axes-level → `my_plot.init()`으로 ax 만들고 그 위에 그리기
 - figure-level(Implot, pairplot 등) → `my_plot` 패턴 쓰지 말고 seaborn이 반환하는 FacetGrid 객체 그대로 다루기
5. 그래프별 필요한 데이터 형태 파악: heatmap→pivot_table, pie→groupby+sum, bar(hue 여러개)→melt, 나머지는 원본 그대로.
6. 처음 써보는 옵션/함수는 **직접 코딩 버전으로 먼저 검증** (셀 하나씩 실행하며 결과 확인) — 한 번에 다 짜지 않기.
7. `x`, `y`로 쓸 컬럼이 수치형인지, `hue`로 쓸 컬럼의 범주 개수(`nunique()`)가 몇 개인지 먼저 확인 (marker/palette 개수 맞추기 위함).
8. scatter/ConvexHull을 쓸 거면 결측치-dtype-hue 범주 개수까지 8-0 체크리스트로 재확인.
9. 검증 끝난 패턴 중 **반복해서 또 쓸 것 같은 것만** 모듈에 함수로 추가 (전부 다 모듈화하려 하지 않기).
10. 모듈(`my_plot.py`) 수정했으면 ****커널 재시작** 또는 `importlib.reload()`로 반영 확인.
11. 그래프 저장 파일명은 데이터셋/그래프 종류별로 겹치지 않게 이름 규칙 정해서 저장.
12. 서브플롯/이중 축을 쓸 거면 `ax=`를 넘기는 순간 title/xlabel/ylabel이 무시된다는 것(12장) 기억하고 필요하면 `ax.set_ylabel()` 등으로 직접 지정.

15. 자주 하는 실수 모음 (통합 오답노트)

데이터 처리 관련

- ❌ 범주형(문자열) 컬럼에 `np.histogram` 사용 → 숫자형 전용 함수. 범주형은 `value_counts()` + `sb.barplot()/sb.countplot()` 사용.
- ❌ `origin.melt()` 인자 없이 호출 → `id_vars`, `value_vars` 안 주면 모든 컬럼이 의도와 다르게 풀림. 항상 명시할 것.

- **✗ melt** 결과를 변수에 저장 안 하고 원본(**origin**)으로 후속 작업 → `origin.groupby("variable")`처럼 존재하지 않는 컬럼 호출 에러. melt 결과는 반드시 새 변수에 저장 후 사용.
- **✗ pivot_table**의 **index**에 값 컬럼 자체를 넣음 → **index**는 그룹 기준이 되는 범주형 컬럼(날짜, 구분 등)이어야 함. **index**=행 기준, **columns**=열 기준, **values**=집계할 값, **aggfunc**=집계함수 4가지 역할 구분해서 쓸 것.
- pandas 메서드 대부분(**drop**, **rename**, **astype** 등)은 원본을 바꾸지 않음 → 결과 유지하려면 재할당하거나 **inplace=True**. (`my_qtcheck.set_type()`이 내부에서 `df = data.copy()`로 명시적 복사 후 처리하는 이유 — 원본 보호.)

복붙 관련 (매우 자주 발생)

- **✗ df1/df2** 복사-붙여넣기 실수 → 코드를 복사해서 재사용할 때 데이터프레임 변수명을 같이 안 바뀌서 엉뚱한 데이터로 그래프가 그려짐.
- **✗ title** 문자열 복사 후 안 바꿈 → "기말고사" 그래프인데 제목은 "중간고사"로 남아있는 등.
- → 복붙 후에는 변수명뿐 아니라 제목 문자열도 항상 다시 확인하는 습관 필요.

파라미터/문법 관련

- **palette**는 반드시 **hue**와 함께 써야 적용됨 (5-2 참고). hue 없이 palette만 쓰면 무시되거나 경고.
- **linewidths(heatmap)** vs **linewidth(line/kde/hist)** — 함수마다 파라미터명 철자가 다름, seaborn 공식 문서 기준으로 확인.
- **markers(raw seaborn, 복수형)** vs **marker(모듈화 함수, 단수형)** — 파라미터명이 함수/버전마다 다를 수 있음.
- **hue** 있는데 **color**도 같이 넘기면 **color**는 무시됨.
- 정의되지 않은 변수(**x** 등)를 그대로 함수에 넘기면 **NameError** — 미완성 코드 남겨두지 않기.

서브플롯/모듈 관련

- **ax**=를 직접 넘기는 함수에서는 **title/xlabel/ylabel/width/height**가 **조용히 무시됨** (12-1 참고, 에러 없이 무시되는 게 제일 위험).
- **twinx=True**는 **rows=1, cols=1**일 때만 작동, 서브플롯 각 칸에 이중 축을 넣는 건 현재 구조에서 미지원 (12-2 참고).
- 같은 이름의 함수를 두 번 정의하면 나중 정의가 앞의 걸 조용히 덮어씀 (에러 없음).
- **import**된 모듈(**my_plot.py**)을 수정해도 이미 실행 중인 커널은 예전 버전을 들고 있음 → **importlib.reload()** 또는 커널 재시작 필요.

Convex Hull 관련

- **hull.vertices**는 인덱스만 줌 → 좌표로 변환(**.iloc[points]**)해서 직접 plot/fill 해야 함.
- 다각형 "닫기" 위해 **np.append(vertices, vertices[0])** 빠뜨리지 않기.
- **plot**(선)과 **fill**(면) 둘 다 그려야 "영역"처럼 보임.
- **outline=True**여도 **hue**가 없으면 자동으로 안 그려짐 (전제 조건).
- ConvexHull 계산 전 **데이터 포인트 3개 이상**인지 반드시 체크.

통계/집계 관련

- 그룹 간 비교 시 이상치 기준은 **그룹별로 따로 계산** (전체 기준으로 한 번에 자르면 그룹 특성이 왜곡될 수 있음).
- 히트맵은 "피벗 테이블을 그린다"는 것과 동의어 — 시각화 전에 반드시 **pivot_table()** 결과 형태(행/열/값)를 먼저 확인.

- Stackplot 텍스트 위치 계산 시 반복문 안에서 `[i]` 인덱싱 빼먹으면 스칼라 대신 시리즈 전체가 섞여 들어감.

16. 실전 템플릿 모음 (바로 복붙용)

16-1. 세션 시작 시 환경 세팅

```
from matplotlib import font_manager as fm
from matplotlib import pyplot as plt
import seaborn as sb
import os, glob as gl
import numpy as np

# 폰트 등록 (fpath는 프로젝트 구조에 맞게 수정!)
fpath = "../helpers/fonts"
font_files = gl.glob(os.path.join(fpath, "*.ttf"))
for f in font_files:
    fm.fontManager.addfont(f)
    fname = fm.FontProperties(fname=f).get_name()
    plt.rcParams['font.family'] = fname

my_dpi = 200
plt.rcParams['font.size'] = 12
plt.rcParams['axes.unicode_minus'] = False
plt.rcParams['figure.dpi'] = my_dpi
plt.rcParams['savefig.dpi'] = my_dpi
plt.rcParams['lines.linewidth'] = 2
plt.rcParams['axes.axisbelow'] = True
```

16-2. 그래프 하나 단독으로 그리기 (my_plot 모듈 기준)

```
fig, ax = my_plot.init(width=1280, height=640, title="제목", xlabel="X축",
                        ylabel="Y축")
sb.lineplot(data=df, x="컬럼1", y="컬럼2", hue="범주컬럼", palette="Set2", ax=ax)
my_plot.show(save_path="myplot.png")
```

16-3. 서브플롯 템플릿

```
fig, ax = my_plot.init(rows=2, cols=3, width=800, height=450, title='제목')
print(f'총 {len(ax)}개의 ax객체가 생성되었습니다.') # 개수 확인 먼저

sb.boxplot(data=df, y='컬럼', ax=ax[0])
sb.histplot(data=df, x='컬럼', kde=True, ax=ax[1])
sb.scatterplot(data=df, x='컬럼A', y='컬럼B', hue='그룹', ax=ax[2])
# ... 필요한 만큼 ax[3], ax[4], ax[5]

my_plot.show()
```

16-4. 이중 축(Twin Axis) 템플릿

```
df_g = origin.groupby('년도').mean()
fig, ax = my_plot.init(title='제목', twinx=True)

my_plot.lineplot(data=df_g, x=df_g.index, y='변수1', color='tab:blue', ax=ax[0])
ax[0].set_ylabel('변수1') # ax= 넘겼으므로 직접 지정 필요!

my_plot.lineplot(data=df_g, x=df_g.index, y='변수2', color='tab:orange', ax=ax[1])
ax[1].set_ylabel('변수2') # 마찬가지로

my_plot.show()
```

16-5. Heatmap 템플릿 (pivot_table + heatmap)

```
pivot_df = pivot_table(df, index="행기준컬럼", columns="열기준컬럼", values="집계할 값", aggfunc=np.mean)
my_plot.heatmap(pivot_df, annot=True, fmt="0.2f", linewidths=0.5, palette="Blues",
                title="제목", xlabel="열기준", ylabel="행기준")
```

16-6. Pie/Donut 템플릿

```
df1 = origin[['범주컬럼', '금액컬럼']].groupby('범주컬럼').sum()
df1.sort_values('금액컬럼', ascending=False, inplace=True)

my_plot.pieplot(x=df1['금액컬럼'], labels=df1.index, autopct="%0.1f%%",
                palette='pastel')
my_plot.pieplot(x=df1['금액컬럼'], labels=df1.index, autopct="%0.1f%%",
                palette='pastel', donutchart=True, wedge_width=0.7)
```

16-7. Bar Plot (wide→long melt 후) 템플릿

```
df_long = melt(origin, id_vars='기준컬럼', var_name='연도', value_name='값')
my_plot.barplot(data=df_long, x="기준컬럼", y="값", hue="연도", palette="Set2",
                title="제목", xlabel="X축", ylabel="Y축")
```

16-8. Scatter + Convex Hull 템플릿

```
my_plot.scatterplot(
    data=origin, x='X컬럼', y='Y컬럼', hue='그룹컬럼',
    size=100, edgecolor="#000000", linewidth=1.5, palette='Set2',
    outline=True, # 군집별 외곽선 + 영역 채우기
    title="제목", xlabel="X축 이름", ylabel="Y축 이름"
)
```

raw로 원리부터 재현할 때:

```
from scipy.spatial import ConvexHull
import numpy as np

def plot_hull(data, x, y, hue, palette, ax):
    classes = list(data[hue].unique())
    for i, v in enumerate(classes):
        df_c = data.loc[data[hue] == v, [x, y]]
        if len(df_c) < 3:
            continue
        hull = ConvexHull(df_c)
        points = np.append(hull.vertices, hull.vertices[0])
        color = sb.color_palette(palette)[i]
        ax.plot(df_c.iloc[points, 0], df_c.iloc[points, 1], linewidth=1,
                linestyle=":", color=color)
        ax.fill(df_c.iloc[points, 0], df_c.iloc[points, 1], alpha=0.1,
                color=color)

fig, ax = my_plot.init()
sb.scatterplot(data=origin, x='X컬럼', y='Y컬럼', hue='그룹컬럼', s=100,
               palette='Set2')
plot_hull(origin, x='X컬럼', y='Y컬럼', hue='그룹컬럼', palette='Set2', ax=ax)
my_plot.show()
```

16-9. LmPlot / PairPlot 템플릿

```
my_plot.lmplot(
    data=origin, x='X컬럼', y='Y컬럼',
    title='제목', xlabel='X축', ylabel='Y축',
    hue='그룹컬럼', palette="Set2", col="분할컬럼",
    markers=["s", "o", "D"],
```

```

scatter_edgecolor="#000000", scatter_linewidths=1, scatter_size=50,
linestyle="--", linecolor="#0066ff", linewidth=2
)

my_plot.pairplot(
    data=origin, hue='그룹컬럼', reg=True, diag_kind='kde',
    title="제목", width=1600, height=1200,
    markers=['o', 's', '^'], palette='Set2',
    scatter_size=5, scatter_alpha=0.7,
    linecolor="#0066ff", linewidth=1, linestyle='--'
)

```

16-10. EDA 표준 루틴 (그래프 그리기 전 필수)

```

origin = load_data('데이터셋이름')
origin.info(); origin.head()

my_qtcheck.set_type(origin, ...) # 타입 정리
my_qtcheck.check_duplicates(origin)
my_qtcheck.check_missing_values(origin)
desc_df = my_qtcheck.numerical_summary(origin) # rel_diff, outliers_ratio,
skew_interpret, log_need 확인
cat_df = my_qtcheck.categorical_summary(origin)

```

17. 색상 팔레트 참고

```

colors = sb.color_palette("Set2", 3) # 3개 색상 리스트로 추출
colors_n = sb.color_palette("Set2", n_colors=len(범주리스트)) # 범주 개수만큼 명시
적으로

```

팔레트 이름	용도
"tab20"	카테고리 많을 때 (최대 20색)
"Set2", "Set3"	파스텔톤, 카테고리 적을 때
"pastel"	부드러운 톤
"Blues", "Reds", "GnBu", "PuRd" 등	heatmap처럼 연속값 강도 표현할 때 (순서형)

hue 넘기면 seaborn이 자동으로 팔레트 적용하지만, bar/barh처럼 직접 반복문 돌리는 경우엔 color_list[i]로 수동 지정 필요. 범주 개수가 팔레트 기본 색상 수(보통 6~10개)를 넘으면 n_colors= 를 명시적으로 지정해야 색이 겹치거나 꼬이지 않음 (Convex Hull, Pie 등에서 특히 중요).

원본 7개 문서(LAB06_데이터시각화_1, _1011, _1213, _2345, 14, 15, _6789) 통합·중복제거본. 각 장의 코드는 실제 노트북에서 검증된 것을 그대로 옮겼으므로 컬럼명만 바뀌어서 바로 재사용 가능.